

AccessPath

Navigation designed
around accessibility, not
just distance.

A mobile navigation concept for wheelchair users. It routes around stairs, steep grades and inaccessible entrances, and it tells you when something breaks partway through your trip.

- ↙ Step-free
- ↙ Low incline
- 🚪 Accessible entrance
- ⚠ Elevator out

DESIGNER	FOCUS	DELIVERABLE	SCOPE
Payton Nguyen	Mobility / wheelchair routing	3 screens + rationale	Concept only, solo

A six-minute route with stairs in it is not a six-minute route

PROBLEM	Navigation apps rank routes by time and distance. For a wheelchair user that ranking can be wrong in a way that really matters, because a route with one flight of stairs in it isn't a slower option, it's not an option. Conditions change on top of that. Elevators break, construction goes up overnight, a side entrance gets locked, and the map still shows the same six minutes.
WHO IT'S FOR	Someone using a wheelchair to get around a campus, neighborhood or transit hub they don't already know. I wanted to be careful about how I framed this. The problem isn't the person's ability. It's that the physical state of a route, whether there's a curb cut or a ramp at the door, is information that exists somewhere and doesn't reach them until they're already standing there.
USER NEED	To know a route will work before committing to it, and to hear about problems early enough to still do something about them.
DESIGN GOAL	Help someone tell whether a route is accessible <i>before</i> they hit an obstacle, and make it easy to change plans when conditions change.
OUT OF SCOPE	The prompt covers mobility, vision and hearing. I picked mobility and went deeper on it instead of covering all three shallowly. Page 06 has where I'd take the other two.

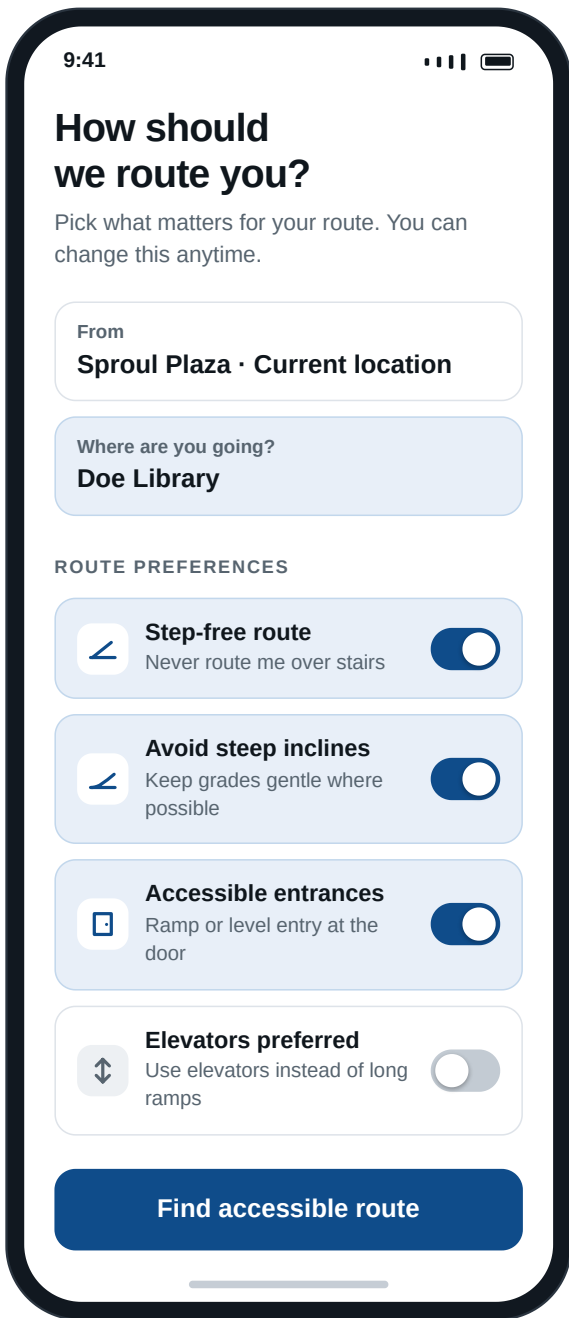
WHAT I ASSUMED, AND WHAT I DIDN'T DO

I didn't run interviews or collect data for this, so nothing on these pages is a research finding. It's my reasoning from the prompt. Three assumptions are doing most of the work:

- **Needs vary.** Two wheelchair users won't want the same route. One avoids every incline, another doesn't care about a gentle one. So the routing has to be adjustable rather than a single "accessible" setting.
- **The data goes stale.** A path that was step-free last month might be blocked today. How recent a report is matters more than how many reports there are.
- **People want to make the call themselves.** If the app hides the faster route, it has quietly decided for them.

What counts as accessible isn't the same for everyone, so the app asks

Before the search runs, the user says what the route should optimize for. Those settings carry through the next two screens.



Screen 1 — preferences. The toggles work; try them.

Settings first, instead of buried in a menu

In most apps accessibility is a toggle somewhere in preferences that you have to know to go looking for. Putting it on the opening screen means the search starts from what the person actually requires, rather than filtering the results afterward.

Every toggle names a physical thing

Stairs, grade, entrances, elevators. I avoided one vague “accessible mode” switch because the user can't see what it's filtering on, and because the app then has no way to explain *why* it rejected a route later.

Three on by default, one off

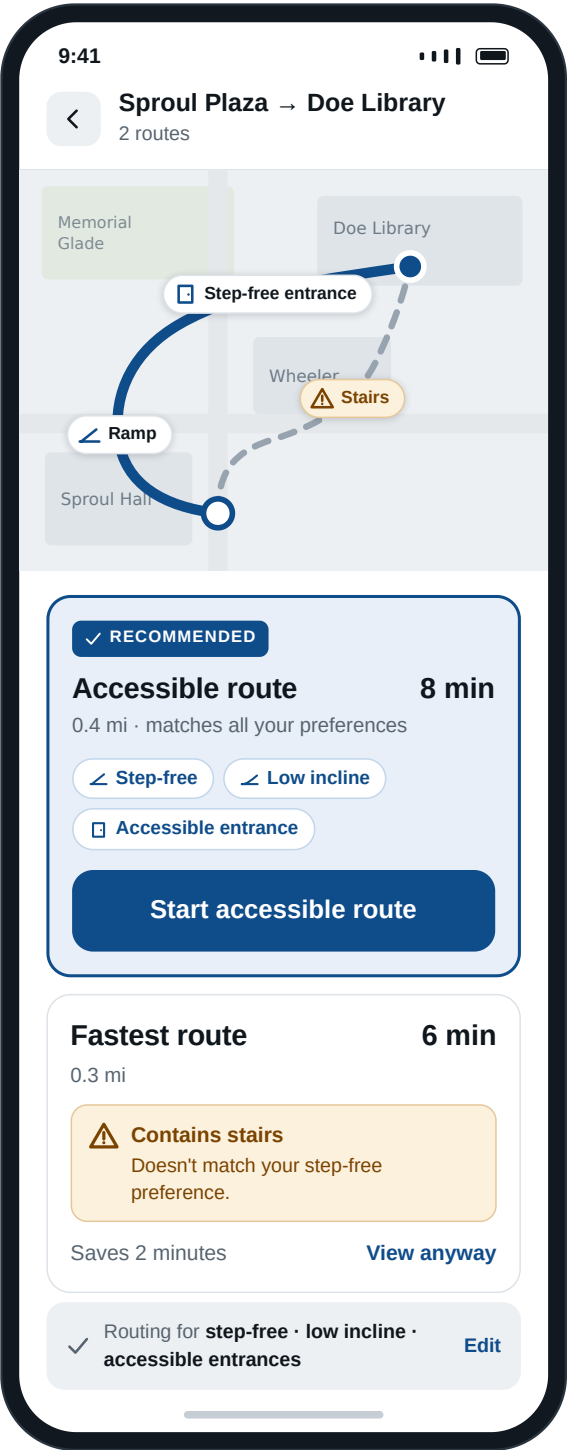
Step-free, inclines and entrances start on because they're the most common needs. Elevators preferred starts off, since some people avoid elevators on purpose. All four can be changed.

The interface has to pass its own test

16 px minimum text, tap targets over 60 px tall, an icon and a label on every row, and on/off shown by the switch position rather than by color.

The recommendation has to explain itself

This is the screen I spent the most time on. The accessible route costs two extra minutes, and the app has to make that worth accepting rather than just asserting it.



Emphasis, rather than deleting the other option

The accessible route gets the filled card, the badge and the only primary button. The fastest route sits right underneath, still perfectly readable. If I removed it the app would be making the decision, and I'd rather it made a recommendation and showed its work.

The warning names something you can check

"Contains stairs. Doesn't match your step-free preference." Not "may not be accessible," which tells you nothing and is the kind of hedge that makes people stop trusting the app. It names the obstacle and points back at the setting from screen 1.

Two minutes, said out loud

Both times are the same size in the same spot, and "Saves 2 minutes" spells out exactly what the faster route buys you. When the cost is this small, saying it plainly is what makes the recommendation easy to accept.

Nothing on the map depends on color

Ramp, step-free entrance and stairs each get an icon and a word. The two route lines are also different weights and one is dashed, so the map still reads if you can't tell the two colors apart.

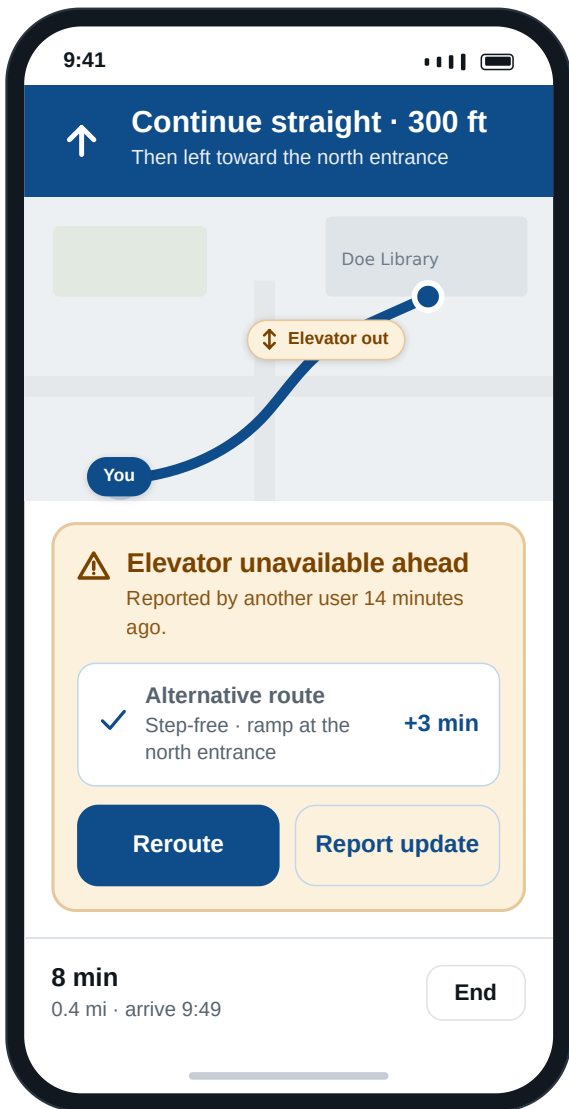
ASSUMPTION

The map here is illustrative. A real version needs sidewalk grade, curb cut and entrance data, which is patchy in most cities. That's the hard part of building this, not the interface.

Screen 2 — route comparison

A route can stop being accessible while you're already on it

Static map data has no way of knowing an elevator went out this morning. The alert needs to arrive while the user still has room to do something about it.



Screen 3 — live obstacle and reroute

How old the report is belongs in the alert

“14 minutes ago” is the reason to act on this. With something temporary, how recent a report is does the work that a star rating would do elsewhere. A week-old report would need much more hedging, or shouldn't interrupt at all.

Price the detour before asking them to take it

+3 min and “step-free” sit on the same card as the Reroute button. Same idea as screen 2. Recommend something, show what it costs, then let the person accept it.

Reporting is a button, not the product

Report update is secondary, and it shows up in context, right when the user is looking at the obstacle and actually knows what's going on. No feed, no profile, no points.

It doesn't take over the screen

The card sits under the turn instruction instead of covering the map, and it can be ignored. Navigation keeps running either way. A warning you have to dismiss before you can see where you're going is its own small obstacle.

Three decisions the whole thing rests on

Personalized routing

People need different things from a route, so the user sets what it optimizes for instead of taking the app's single built-in idea of what "accessible" means.

Accessibility over speed

The app explains why the route it picked is longer, and leaves the faster one on screen with a specific reason attached. The choice stays with the person making the trip.

Real-time information

Recent reports cover what static map data misses. Broken elevators, construction, entrances that got closed this morning.

WHAT I CONSIDERED AND REJECTED

One "accessible mode" switch

Fewer taps than four toggles. But it hides what's being filtered on, and one switch can't tell you why a route got rejected.

Four named settings cost one screen and are what make the warning on screen 2 possible.

Hiding inaccessible routes

A shorter, cleaner list. But it takes the judgment away from the user. Some people would go up two steps with a hand, some wouldn't.

Show it, name the obstacle, let them choose.

A full crowdsourced access map

A bigger idea, but it only works if enough people in one place are reporting, and it makes the product about contributing instead of about getting somewhere.

Reporting stays one button, inside navigation.

HOW THE UI APPLIES THIS TO ITSELF

- **An icon and a word.** No status is carried by color on its own.
- **Big targets and big type.** 16 px minimum body text, tap targets over 44 px.
- **Plain words.** "Contains stairs," not "limited accessibility."
- **High contrast, not much clutter.** One accent color, one warning color, no gradients.

WHERE I'D TAKE IT NEXT

- **Vision.** Spoken directions cued to landmarks and surface changes rather than distances.
- **Hearing.** Visual and haptic transit alerts for stop and platform changes.
- **Verified venues.** Entrance and elevator data confirmed by the building operator, sitting alongside user reports.
- **The actual blocker.** Getting the sidewalk grade, curb cut and entrance data in the first place, and getting enough people reporting in one place for the live updates to be worth anything.

This is a concept, not a working product. I didn't do user research for it, so everything here is my own reasoning from the prompt, and I've labeled the assumptions on page 02. The next thing I'd actually do is walk a route like this one with someone who uses a wheelchair and find out where I got it wrong.